

C Programming (W5)



Welcome!!

Please check attendance individually.
(Mobile App)

Things to do today

01 | Lecture Notes (Ch.5 ~)

02 | Operators

03 |

Terminology

Expression:

An **expression** is a combination of variables, constants, operators, and function calls that are evaluated to produce a value.

Example: **a + b** is an expression.

Operation:

An **operation** is the action performed by an operator on one or more operands.

Example: In **a + b**, the **+** operator performs an addition operation.

Statement:

Complete instruction that tells the computer to do something.

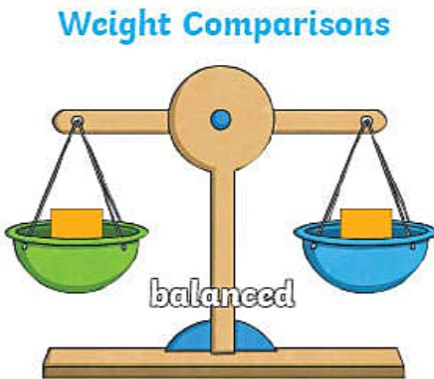
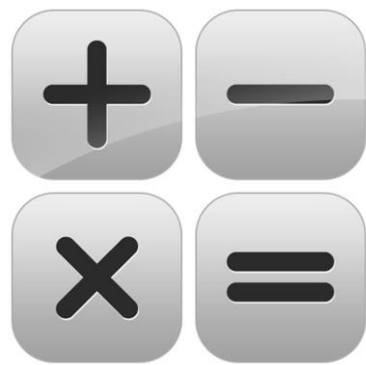
Example: **int sum = a + b;**

Operators

In programming, an **operator** is a symbol or keyword that tells the computer to perform a specific action on data (values).

[Common Types of Operators]

1. Arithmetic Operators (for calculations)
2. Comparison Operators (for comparing values)



3. Logical Operators (for combining conditions)
4. Assignment Operator (for storing values)

Input		Output
A	B	
0	0	0
0	1	0
1	0	0
1	1	1

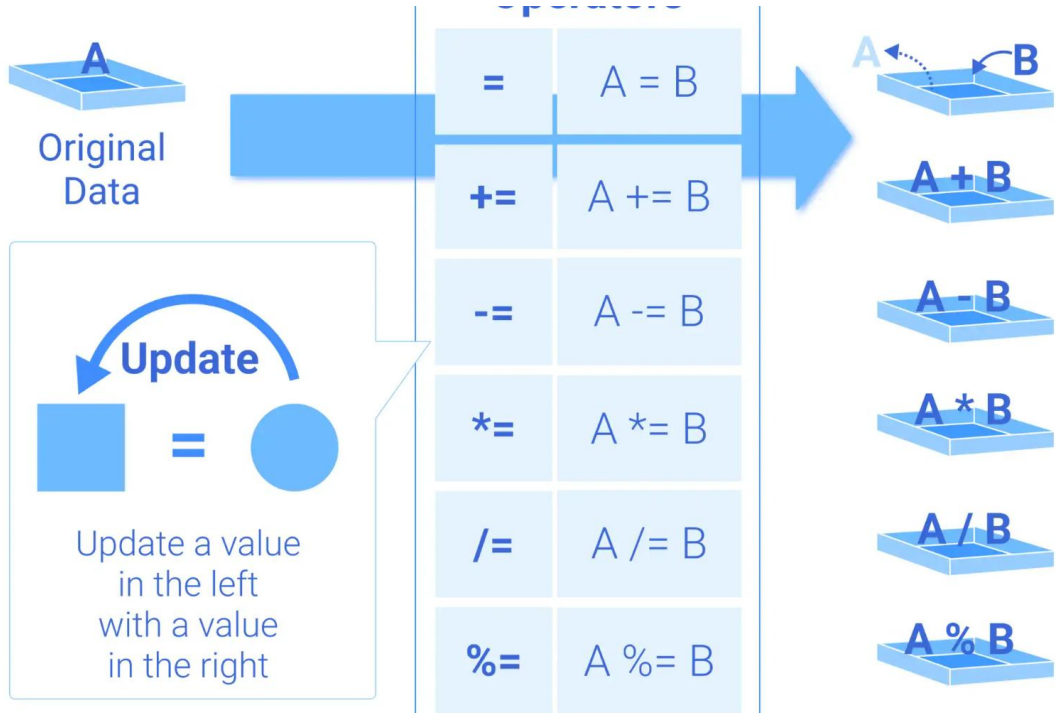
AND

Input		Output
A	B	
0	0	0
0	1	1
1	0	1
1	1	1

OR

Input	Output
A	
0	1
1	0

NOT



Operators

1. Arithmetic Operators

Arithmetic Operators: Used to perform basic mathematical operations.

Ex: **+**, **-**, *****, **/**, **%** etc

2. Relational (Comparison) Operators

Relational (Comparison) Operators: Used to compare two values.

Ex: **==**, **!=**, **>**, **<**, **>=**, **<=** etc

3. Logical Operators

Logical Operators: Used to combine or invert boolean expressions.

Ex: **&&**, **||**, **!** etc

4. Assignment Operators

Assignment Operators: Used to assign values to variables.

Ex: **=**, **+=**, **-=**, ***=**, **/=**, **%=** etc

Operators

1. Arithmetic Operators

Used to perform basic mathematical operations.

Operator	Description	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulus (remainder)	$a \% b$

2. Relational (Comparison) Operators

Used to compare two values.

Operator	Description	Example
==	Equal to	$a == b$
!=	Not equal to	$a != b$
>	Greater than	$a > b$
<	Less than	$a < b$
>=	Greater or equal	$a >= b$
<=	Less or equal	$a <= b$

Operators

3. Logical Operators

Used to combine or invert boolean expressions.

Operator	Description	Example
&&	Logical AND	a && b
!	Logical NOT	!a

4. Assignment Operators

Used to assign values to variables.

Operator	Description	Example
=	Assign	a = b
+=	Add and assign	a += b
-=	Subtract and assign	a -= b
*=	Multiply and assign	a *= b
/=	Divide and assign	a /= b
%=	Modulus and assign	a %= b

Operator

- + (Addition)
- (Subtraction)
- * (Multiplication, Asterisk)
- / (Division)
- % (Modulus / Remainder)
- = (Assignment)
- == (Equal to, Comparison)
- != (Not equal to)
- > (Greater than)
- < (Less than)
- >= (Greater than or equal to)
- <= (Less than or equal to)
- && (Logical AND)
- || (Logical OR)
- ! (Logical NOT)

- & (Bitwise AND / Address-of operator, Ampersand)
- | (Bitwise OR)
- ^ (Bitwise XOR)
- ~ (Bitwise Complement)
- << (Left shift)
- >> (Right shift)
- += (Addition assignment)
- = (Subtraction assignment)
- *= (Multiplication assignment)
- /= (Division assignment)
- %= (Modulus assignment)
- &= (Bitwise AND assignment)
- |= (Bitwise OR assignment)
- ^= (Bitwise XOR assignment)
- <<= (Left shift assignment)
- >>= (Right shift assignment)
- ++ (Increment)
- (Decrement)
- > (Structure pointer access)
- . (Structure member access)
- ? : (Ternary conditional operator)
- , (Comma operator)

See you next week!

DO NOT miss the classes



용어

Expression (식):

식은 변수, 상수, 연산자, 함수 호출 등을 조합하여 하나의 값을 생성하는 코드 조각입니다.

예시: $a + b$ 는 하나의 식입니다.

Operation (연산):

연산은 연산자가 하나 이상의 피연산자에 대해 수행하는 동작입니다.

예시: $a + b$ 에서 $+$ 연산자는 덧셈 연산을 수행합니다.

Statement (문):

문은 컴퓨터에게 어떤 작업을 수행하라고 지시하는 완전한 명령입니다.

예시: `int sum = a + b;`는 하나의 문입니다.

Operators

1. Arithmetic Operators (산술 연산자)

기본적인 수학 연산을 수행할 때 사용됩니다.

예: **+**, **-**, *****, **/**, **%** 등

2. Relational (Comparison) Operators (관계(비교) 연산자)

두 값을 비교할 때 사용됩니다.

예: **==**, **!=**, **>**, **<**, **>=**, **<=** 등

3. Logical Operators (논리 연산자)

불리언(boolean) 표현식을 결합하거나 반전시킬 때 사용됩니다.

예: **&&**, **||**, **!** 등

4. Assignment Operators (대입 연산자)

변수에 값을 할당할 때 사용됩니다.

예: **=**, **+=**, **-=**, ***=**, **/=**, **%=** 등

연산

1. Arithmetic Operators

Arithmetic Operators: Used to perform basic mathematical operations.

Ex: **+**, **-**, *****, **/**, **%** etc

2. Relational (Comparison) Operators

Relational (Comparison) Operators: Used to compare two values.

Ex: **==**, **!=**, **>**, **<**, **>=**, **<=** etc

3. Logical Operators

Logical Operators: Used to combine or invert boolean expressions.

Ex: **&&**, **||**, **!** etc

4. Assignment Operators

Assignment Operators: Used to assign values to variables.

Ex: **=**, **+=**, **-=**, ***=**, **/=**, **%=** etc

Statement

In **C**, a **statement** is a complete instruction that tells the computer to do something.

It usually ends with a **semicolon (;)**. Compound Statement / Block. { }

Types of Statements in C:

1. **Declaration Statement:** Used to declare variables.

```
int a;  
float pi = 3.14;
```

2. **Control Flow Statements:** Control the flow of execution.

- **Conditional statement:**

```
if (a > b) { ... }  
else {...}
```

- **Iteration Statement (Looping):**

```
for (int i = 0; i < 10; i++) { ... }  
while() {...}
```

3. **Jump Statements:** Change the flow directly.

```
break;  
continue;  
return;  
goto
```

Statement

C에서 문장은 컴퓨터에게 어떤 작업을 하라고 지시하는 완전한 명령어.

일반적으로 세미콜론(;)으로 끝남.

복합 문장/블록은 {} 사용.

C의 명령문 유형:

1. 선언문: 변수를 선언하는 데 사용.

```
int a;
```

```
float pi = 3.14;
```

2. 제어 흐름 문: 실행 흐름을 제어.

- 조건문

```
if (a > b) { ... }
```

```
else {...}
```

- 반복문

```
for (int I = 0; I < 10; i++) { ... }
```

```
while() {...}
```

3. 점프문: 흐름을 직접 변경.

```
break;
```

```
continue;
```

```
return;
```

```
goto
```

Notice – Modify what I mentioned

Build Process (*The build process consists of compilation and linking.*)

- Compilation

- . Translating source code (.c files) into object code (.o or .obj files).
- . Each source file is compiled independently by the compiler.

- Linking

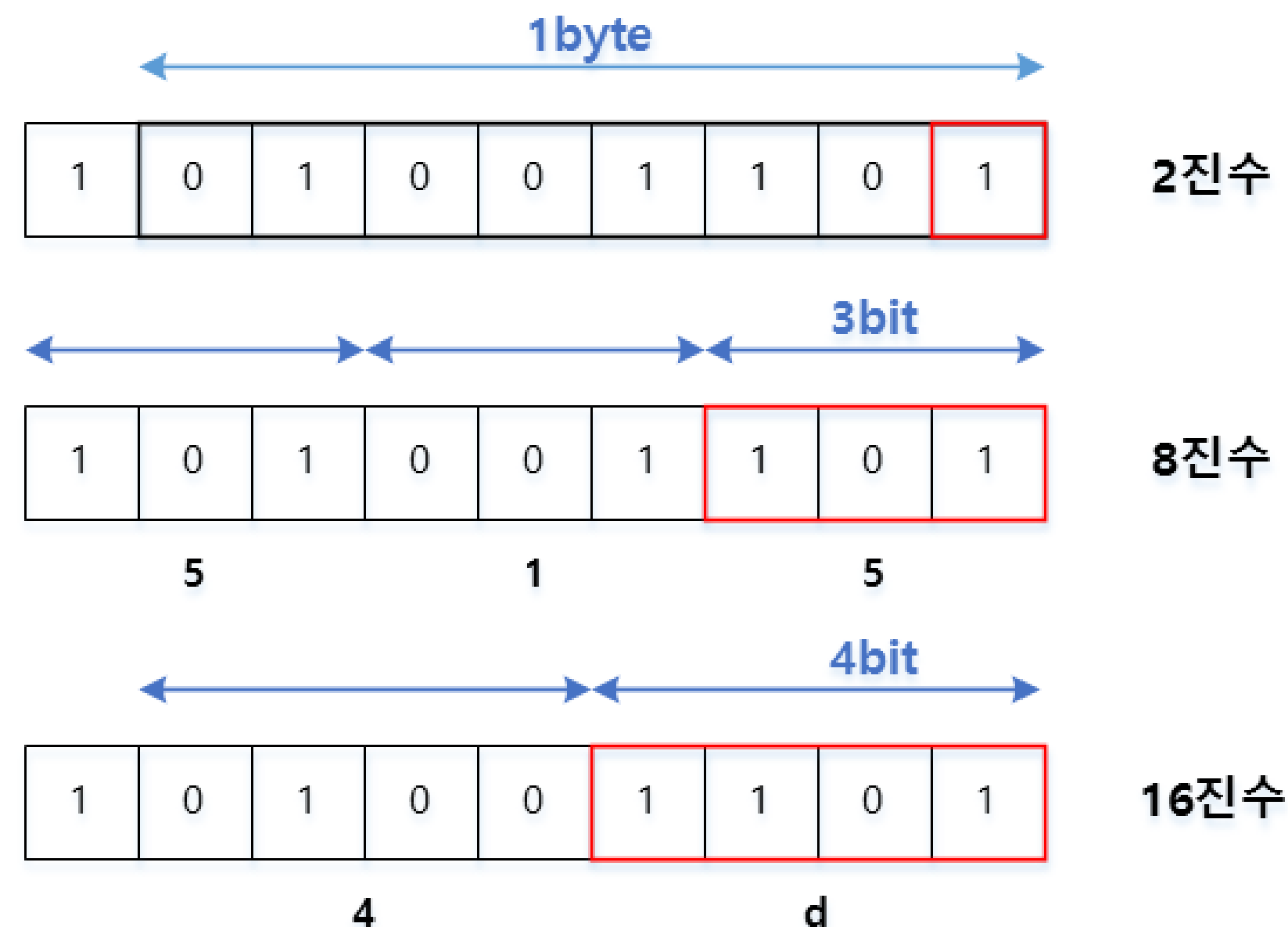
- . Combining object files and libraries into a final executable (.exe on Windows, no extension on Linux).
- . Resolves external references (e.g., function calls, global variables).

The entire workflow = Compilation + Linking (and sometimes preprocessing, optimization, packaging).

General knowledge

Binary → 2 base / **Octal** → 8 base / **Decimal** → 10 base / **Hexadecimal** → 16 base

2진수	10진수	16진수	8진수
0000 0000	0	0x0	00
0000 0001	1	0x1	01
0000 0010	2	0x2	02
0000 0011	3	0x3	03
0000 0100	4	0x4	04
0000 0101	5	0x5	05
0000 0110	6	0x6	06
0000 0111	7	0x7	07
0000 1000	8	0x8	010
0000 1001	9	0x9	011
0000 1010	10	0xa	012
0000 1011	11	0xb	013
0000 1100	12	0xc	014
0000 1101	13	0xd	015
0000 1110	14	0xe	016
0000 1111	15	0xf	017
0001 0000	16	0x10	020
0001 0001	17	0x11	021



중요

빌드 프로세스 (빌드 프로세스는 컴파일과 링크로 구성)

- 컴파일(Compilation):
 - . 소스 코드(.c 파일)를 오브젝트 코드(.o 또는 .obj 파일)로 변환한다.
 - . 각 소스 파일은 컴파일러에 의해 독립적으로 컴파일된다
- 링크(Linking) :
 - . 오브젝트 파일과 라이브러리를 결합하여 최종 실행 파일을 생성한다.
(Windows에서는 .exe, Linux에서는 확장자 없음)
 - . 외부 참조(예: 함수 호출, 전역 변수)를 해결합니다.

전체 워크플로 = 컴파일 + 링크 (그리고 경우에 따라 전처리, 최적화, 패키징).

연산자

1. 산술 연산자

기본적인 수학 연산을 수행하는 데 사용됩니다..

Operator	Description	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulus (remainder)	$a \% b$

2. 관계(비교) 연산자

두 값을 비교하는 데 사용됩니다.

Operator	Description	Example
==	Equal to	$a == b$
!=	Not equal to	$a != b$
>	Greater than	$a > b$
<	Less than	$a < b$
>=	Greater or equal	$a >= b$
<=	Less or equal	$a <= b$

연산자

3. 논리 연산자

부울 표현식을 결합하거나 반전하는 데 사용됩니다.

- 부울 표현식(Boolean expression)은

0(거짓)과 1(참)의 논리값, 변수, 그리고 논리 연산자(AND, OR, NOT)를 결합하여 만든 논리식을 의미하며, 최종 결과값은 0 또는 1이 됩니다

Operator	Description	Example
&&	Logical AND	a && b
,		,
!	Logical NOT	!a

4. 대입 연산자

변수에 값을 할당하는 데 사용됩니다.

Operator	Description	Example
=	Assign	a = b
+=	Add and assign	a += b
-=	Subtract and assign	a -= b
*=	Multiply and assign	a *= b
/=	Divide and assign	a /= b
%=	Modulus and assign	a %= b

특수 문자

{ } (중괄호 - 코드 블록에 사용)

[] (대괄호 - 배열에 사용)

() (괄호 - 함수 및 표현식에 사용)

; (세미콜론 - 문장 종료 문자)

: (콜론 - 레이블 및 삼항 연산자에 사용)

(해시 - 전처리기 지시어)

" (큰따옴표 - 문자열 리터럴)

' (작은따옴표 - 문자 리터럴)

**** (백슬래시 - 이스케이프 시퀀스)

// (한 줄 주석)

/* */ (여러 줄 주석)

! (느낌표) NOT 연산자

연산자

+ (덧셈)
 - (뺄셈)
 * (곱셈, 별표)
 / (나눗셈)
 % (나머지)
 = (부호)
 == (같음, 비교)
 != (같지 않음)
 > (보다 큼)
 < (보다 작음)
 >= (크거나 같음)
 <= (작거나 같음)
 && (논리 AND)
 || (논리 OR)
 ! (논리 NOT)

& (비트 AND / 주소 연산자, 앰퍼샌드)
 | (비트 OR)
 ^ (비트 XOR)
 ~ (비트 보수)
 << (좌측 시프트)
 >> (우측 시프트)
 += (덧셈)
 -= (뺄셈)
 *= (곱셈)
 /= (나눗셈)
 %= (나머지 연산)
 &= (비트 AND)
 |= (비트 OR)
 ^= (비트 XOR)
 <<= (좌측 시프트)
 >>= (우측 시프트)
 ++ (증가)
 -- (감소)
 -> (구조체 포인터 접근)
 . (구조체 멤버 접근)
 ?: (삼항 조건 연산자)
 , (쉼표 연산자)